

## **Custom Vertex Shader and Procedural Terrain**

The terrain is a 40x40 plane geometry that is subdivided into 150x150 segments, and is then rotated to be horizontal. We then used a shader material to displace each vertex's height using `uTime` and `uAmplitude` uniforms. The height is a sum of 3 sine/cosine waves that travel along the x, z, and diagonal axes, with each having a different frequency and speed. The displaced position then goes through the full Model View Projection pipeline. The vertex shader also passes the normalized height to the fragment shader, and the fragment shader blends 5 elevation colors (dark blue, light blue, green, grey, and white, representing the colors of the earth) using `smoothstep` and `mix`. For the highlight, we used a `uSatellitePosition` uniform that is updated with the satellite's position every frame, and `smoothstep` on each vertex's distance to the satellite to scale up and tint orange the terrain beneath it. The biggest challenge here was writing the vertex and fragment shaders and figuring out how to properly incorporate the MVP pipeline.

## **Satellite Transformations and Hierarchical Scene Graph**

The satellite in our project is built using a hierarchy of different objects and components, such as the main body, solar panel rods, solar panels, and the front-facing sphere. All of these components are combined in a group, allowing all of them to have the same transformations. For the transformations, we use real-world time in a `deltaTime` variable. The position is constantly being updated with  $x=r*\cos(\theta)$  and  $y=r*\sin(\theta)$ , which are formulas to make the satellite rotate around a circle of radius  $r$ . Using the `lookAt` function, we ensure the front of the satellite is always pointing towards the middle of the terrain. Lastly, the solar panels additionally rotate independently of the satellites rotation simply by updating `rotation.x` with `deltaTime`.

### **Camera Viewing Models and Projection Mechanics:**

The project uses both perspective and orthographic cameras. The perspective camera uses a 60-degree field of view near and far clipping planes of 0.1 and 1000, creating realistic depth by making distant objects appear smaller. The orthographic camera provides a top-down view without perspective distortion. Users can press 1 to use the perspective camera and 2 to switch to the orthographic camera. The perspective camera supports WASD movement relative to the direction it is facing. The forward direction is calculated with `getWorldDirection()`, and movement is multiplied by `deltaTime` so the speed remains consistent across different frame rates. Camera movement is also limited to  $\pm 50$  units along the X and Z axes. Mouse movement controls the camera using yaw and pitch. Pitch is limited to prevent the camera from flipping, and Pointer Lock allows continuous rotation without the cursor being stopped by the edge of the browser window. When the browser window is resized, both cameras update their projection settings. The active camera is then used to render the scene with `renderer.render(scene, activeCamera)`.

### **Above and Beyond**

For the above and beyond, we chose to implement particle effects for our satellites thrusters. We've done this by creating the particles with buffer geometry, and then made arrays for each particle's position and velocity. The particle's direction was done by taking the negative of the tangent line of the satellite's direction, so the particles would realistically look like they are thrusting the satellite. Velocities were calculated using the direction of the particles, which made us able to calculate new positions of the particle and constantly update them on the renderer. If a particle's position is too far away from the satellite, it will get reset back to its origin position (the satellite). The biggest challenge with this was the calculation of the particle's direction with it constantly changing.